# Übungsblatt 10

## Dependency Injection, Testing und Persistenz

5430UE Web and Data Engineering

24. Juni 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

# Lernziele

- Grundlegendes Verständnis der Konzepte Inversion of Control (IoC) und Dependency Injection (DI) im Spring Framework.
- Einblick in verschiedene Testarten sowie die Test-Pyramide.
- Grundlegendes Verständnis der Dateninitialisierung beim Anwendungsstart und der Vererbungsabbildung mit JPA.

# Dependency Injection (DI) und Inversion of Control (IoC)

# Dependency Injection (DI) und Inversion of Control (IoC) (1/3)

Eines der Kernkonzepte des Spring Frameworks ist die *Dependency Injection* (DI) als konkrete Umsetzung des *Inversion of Control* (IoC) Prinzips.

1. Erklären Sie in eigenen Worten, was man unter Dependency Injection versteht und wie das Konzept grundlegend funktioniert. Gehen Sie dabei kurz auf die Begriffe *Inversion of Control (IoC)*, *IoC-Container* und *Spring Bean* ein.



# Dependency Injection (DI) und Inversion of Control (IoC) (2/3)

Grundlage für die folgenden beiden Teilaufgaben stellen die nachfolgend gegebenen Implementierungen eines `OrderService` dar.

```
// Version A
@Service
public class OrderService {
  private final EmailService email = new EmailService();
  public void placeOrder(Order o) { email.send(o); }
}

// Version B
@Service
public class OrderService {
  private final EmailService email;

  public OrderService(EmailService email) {
    this.email = email;
  }

  public void placeOrder(Order o) { email.send(o); }
}
```



# Dependency Injection (DI) und Inversion of Control (IoC) (3/3)

2. Welche der beiden Versionen ist besser testbar? Begründen Sie Ihre Antwort.

3. Was macht Spring im Hintergrund automatisch bei Version B, wenn die Anwendung gestartet wird?

# Die Test-Pyramide

# Die Test-Pyramide

Beschreiben Sie die drei Ebenen der Test-Pyramide für eine (Spring Boot) Anwendung.

1. Was testet jede Ebene? Nennen Sie ein konkretes Beispiel.
2. Wie verhält sich die empfohlene Anzahl der Tests sowie deren Ausführungsgeschwindigkeit von der untersten zur obersten Ebene?

# Datenbank-Initialisierung mit dem CommandLineRunner



# Datenbank-Initialisierung mit dem CommandLineRunner

Bei der Entwicklung einer Spring Boot-Anwendung ist es oft hilfreich, wenn beim Start automatisch Testdaten in die (In-Memory-)Datenbank geladen werden, damit man direkt mit der Entwicklung der Endpunkte oder der UI beginnen kann.

Laden Sie sich dazu das Projekt `CommandLineRunnerDemo.zip` aus Stud.IP herunter. Das Projekt enthält bereits eine fertige Entität `Product` (mit Name und Preis) sowie das zugehörige `ProductRepository`.

1. Was ist ein `CommandLineRunner` in Spring Boot und zu welchem exakten Zeitpunkt im Lebenszyklus der Anwendung wird er ausgeführt?
2. Im Projekt finden Sie die leere Klasse `DataInitializer`. Erweitern Sie diese Klasse so, dass sie beim Starten der Anwendung automatisch drei verschiedene Produkte in der Datenbank speichert. (Anforderungen an den Code siehe Folgeseite)



# Anforderungen an die Code-Implementierung

- Nutzen Sie Dependency Injection, um das `ProductRepository` einzubinden.
- Geben Sie unmittelbar vor und nach dem Speichern der Daten jeweils eine Meldung über `System.out.println(...)` auf der Konsole aus (z. B. „*Starte Initialisierung...*“), damit Sie den Zeitpunkt der Ausführung in den Start-Logs von Spring gut erkennen können.



# JPA Vererbungsstrategien

# JPA Vererbungsstrategien

Sie modellieren eine Klassenhierarchie für das Personalwesen einer Hochschule: Eine abstrakte Basisklasse `Person` (Attribute: `id`, `name`), von der die Klassen `Student` (zusätzlich: `semester: int`) und `Lehrkraft` (zusätzlich: `lehrstuhl: String`) erben.

1. Skizzieren Sie die drei JPA-Vererbungsstrategien (`SINGLE_TABLE`, `JOINED`, `TABLE_PER_CLASS`). Erklären Sie jeweils in einem Satz, wie die Tabellenstruktur aussieht, nennen Sie Vor- und Nachteile und geben Sie an, wie die Annotation an der Basisklasse `Person` aussehen muss.

2. Welche Strategie würden Sie für dieses konkrete Szenario wählen und warum?

# Vertiefung: Die Test-Pyramide in der Praxis (Hands-On)

# Vertiefung: Die Test-Pyramide in der Praxis (Hands-On) (1/2)

*Hinweis: Diese Aufgabe wird in der Übung je nach zur Verfügung stehender Zeit besprochen.*

Laden Sie sich das Projekt `testdemo.zip` aus Stud.IP herunter und öffnen Sie es in Ihrer IDE.

**Aufbau des Projekts:** Das Projekt simuliert ein stark vereinfachtes Immatrikulationssystem.

- `Student` hält die Daten (hier nur den Namen).
- `StudentService` enthält die Geschäftslogik und prüft, ob der Name gültig ist.
- `StudentController` nimmt HTTP-POST-Anfragen entgegen und ruft den Service auf.

# Vertiefung: Die Test-Pyramide in der Praxis (Hands-On) (2/2)

Die zugehörigen Testklassen finden Sie im Projektbaum unter `src/test/java/wde/testdemo`. Die Tests sind lauffähig, jedoch fehlen Erklärungen.

1. **Klassifizierung:** Ordnen Sie die drei Testklassen (`StudentTestA`, `StudentTestB`, `StudentTestC`) der korrekten Ebene der Testpyramide zu (Unit Test, Integration Test, E2E Test). Begründen Sie Ihre Zuordnung kurz anhand der im Code verwendeten Annotationen oder fehlenden Spring-Kontexte.
2. **Code-Verständnis:** Analysieren Sie den Quellcode der drei Testklassen. Ergänzen Sie aussagekräftige (Inline-)Kommentare direkt im Quellcode, die erklären, *was* in den markanten Zeilen passiert (insbesondere bei den Annotationen und Mocking-Befehlen).

# Materialien zum Übungsblatt

- Aufgabe *Datenbank-Initialisierung mit dem CommandLineRunner*: Vorlage CommandLineRunner (`CommandLineRunnerDemo.zip`)
- Aufgabe *Die Test-Pyramide in der Praxis (Hands-On)*: Vorlage Test-Pyramide (`testdemo.zip`)